

Station ALMA-7, Part III: The Repair Fleet

Encapsulation · Inheritance · Polymorphism · Protocols · Extensions

Six weeks after the teleporter was fixed. The hull needs constant patching, so the station now runs a fleet of small repair drones: welders, scanners and cargo haulers. Each is a different machine, but mission control wants one button that says "run the shift" and one screen that says "what's broken".

The previous engineer solved this with copy-paste. There are three nearly identical shift loops, the battery charge is a public variable anyone can set to 100, and the diagnostics screen can only show drones — the hull sensors and the old beacon from the original station don't fit in the array, so nobody has checked them in two years.

You are rebuilding it properly. One base class for what all drones share. One protocol for anything that can report its health, whether it's a class, a struct, or a type you are not allowed to modify.

Topics: encapsulation, inheritance, overriding, `super`, `final`, polymorphism, protocols (property, method, mutating requirements), protocols as types, protocol extensions, extensions.

Submission: your completed `ALMA7_Part3_Starter.swift` (or a Playground with the same contents), with answers to the defense questions in the comment block at the end.

Station Rules

1. **! (force unwrap) is forbidden.** −0.5 points each. The Incident Reports section is the only exception.
2. No `map`, `filter`, `reduce`, `compactMap`. Write the loops.
3. Use the **exact** type, protocol and method names given here.
4. **No duplicated logic.** If the same rule appears in two types, you lost the point of this module. Every repeated rule belongs in a base class or a protocol extension, written once.
5. Drone types are classes. Everything else should be a struct unless you can justify otherwise.

Starter File

Download `ALMA7_Part3_Starter.swift` and paste its contents into a Playground (or run `swift ALMA7_Part3_Starter.swift`). **Do not modify the starter section** — that restriction is the whole point of Level 5.

Item	What it is
<code>fleetData</code>	6 drone records: <code>(kind: String, id: String, charge: Int)</code> . One <code>kind</code> is not a real drone
<code>sensorData</code>	2 hull sensors: <code>(id: String, charge: Int)</code>
<code>LegacyBeacon</code> + <code>beacon</code>	A type from the original station. You may not edit it. No methods, no protocols, just two properties
Solution section	Commented-out declarations, in order
Incident reports	The broken code for Level 6

The Health Rule

One rule, used by every component in the station:

Charge / signal	<code>statusCode</code>	Meaning
below 20	2	critical
20 to 49	1	warning
50 and above	0	nominal

This rule must exist in **exactly one place** in your file. If you write these thresholds twice, that's −0.5 points, no matter how

correct the output is.

Level 1 · The Power Cell (encapsulation, 1.5 points)

The old code had `var charge = 100` sitting in the open, and three different files setting it directly.

```
final class PowerCell {
    private var charge: Int
}
```

`charge` is `private`. Everything outside the type goes through methods:

```
init(charge: Int)           // clamps the starting value into 0...100
func level() -> Int          // read-only access
func spend(_ amount: Int) -> Bool
func recharge(by amount: Int)
```

- `spend` returns `false` and changes nothing if `amount` is zero or negative, or if there isn't enough charge. Otherwise it subtracts and returns `true`.
- `recharge(by:)` ignores non-positive amounts and never goes above 100.

Then prove the encapsulation works: write `cell.charge = 100` outside the class, run it, and leave the line commented out with the exact compiler error.

Why a class and not a struct here? Answer in a comment above the type. One sentence.

Level 2 · The Fleet (inheritance, 2 points)

Task 2.1 — The base class

```
class Drone {
    let id: String
    let cell: PowerCell

    init(id: String, cell: PowerCell)

    var powerCost: Int { 10 }           // overridable
    var statusLine: String              // "W-1: 80%"
    func performTask() -> Int { 0 }     // work units produced; a bare drone does nothing

    final func runOnce() -> Int         // see below
}
```

`runOnce()` is the shift ritual, and it is `final` on purpose:

- Try to spend `powerCost` from the cell.
- If that fails, return `0` and do nothing else.
- Otherwise return `performTask()`.

Subclasses change the **cost** and the **work**, never the ritual. Write one sentence in a comment explaining what `final` buys you here.

Task 2.2 — The three drones

Type	powerCost	performTask()	Extra
WelderDrone	25	40	<code>func weldSeam() -> String</code> , and the class is <code>final</code>
ScannerDrone	10	15	overrides <code>statusLine</code> to append " [scanner]" — and must call <code>super</code> rather than rebuilding the string
CargoDrone	20	25	<code>final</code> class

Task 2.3 — The factory

```
func makeDrone(kind: String, id: String, charge: Int) -> Drone?
```

Returns the right subclass for "welder", "scanner", "cargo", and `nil` for anything else. Build the fleet from `fleetData` and print a warning for the record you had to skip.

Note the return type: `Drone?`, not `WelderDrone?`. A function that returns a superclass can hand back any subclass — that's the whole idea you'll use in Level 3.

Level 3 · The Shift (polymorphism, 1.5 points)

```
func runShift(_ fleet: [Drone], rounds: Int) -> Int
```

Runs `rounds` passes over the fleet, calling `runOnce()` on every drone, and returns the total work units produced. The function knows nothing about welders or scanners — one loop, one call, different behaviour per object. That is polymorphism, and the whole reason the base class exists.

Run the shift with `rounds: 3` and print, after it finishes:

- every drone's `statusLine`
- how many drones still have enough charge for one more task

Mission fragment A = total work units from the 3-round shift. **Mission fragment B** = the sum of all drone charge levels after the shift. **Mission fragment C** = how many drones could still run one more task.

A drone whose battery is too low must simply produce nothing — no crash, no negative charge.

Level 4 · Diagnostics (protocols, 2 points)

Task 4.1 — Diagnosable

```
protocol Diagnosable {
    var componentID: String { get }
    var statusCode: Int { get }
    func diagnose() -> String
}
```

Task 4.2 — Rechargeable

```
protocol Rechargeable {
    mutating func recharge(by amount: Int)
}
```

Conform **both**:

- `Drone` — a class. Note that you write the method **without** `mutating`. Explain why in a comment.
- `SensorModule` — a `struct` you write yourself, with `var chargeLevel: Int`, built from `sensorData`. Its `recharge(by:)` **is** `mutating`.

This is the point of the module: a protocol unites a class and a struct, something inheritance can never do.

Task 4.3 — The diagnostics screen

```
func diagnosticsReport(_ components: [Diagnosable]) -> String
```

Build one array holding **all** your drones *and* your sensor modules at once, pass it in, and print the report. The array's element type is a protocol, not a class — write one comment sentence saying why `[Drone]` could never have held the sensors.

Level 5 · Shared Behaviour (protocol extensions and extensions, 1.5 points)

Task 5.1 — The rule lives in the protocol

Write an extension on `Diagnosable` providing:

- a default `diagnose()` that returns `"<componentID>: code <statusCode>"`;
- a helper that turns a charge level into a status code, so the **Health Rule** table above is implemented exactly once in your whole file. Every conforming type calls this helper from its `statusCode`.

Task 5.2 — The beacon you can't edit

`LegacyBeacon` is in the untouchable part of the starter file. Make it `Diagnosable` **without changing its declaration**, using

an extension. Its status comes from `signalStrength` and the same rule.

Give the beacon a custom `diagnose()` that returns something clearly different from the default — the maintenance crew needs to spot legacy hardware in the log.

Add `beacon` to your diagnostics array and print the report again.

Mission fragment D = the sum of `statusCode` over **all** components: drones (after the shift), sensors, and the beacon.

Task 5.3 — Extending a type you didn't write

```
extension Int {  
    var powerBar: String // 42 -> "####....."  
}
```

Ten characters: one `#` per full 10 units, `.` for the rest, clamped to `0...10` so that `-5` and `250` don't break it. Use it in `statusLine`.

Level 6 · Incident Reports (bug hunt, 1 point)

Four excerpts are in your starter file. **Two don't compile. Two compile and lie.**

```
// Report 1  
class PatchDrone: Drone {  
    func performTask() -> Int {  
        return 30  
    }  
}  
  
// Report 2  
final class HeavyWelder: WelderDrone {  
    override func runOnce() -> Int {  
        return 999  
    }  
}  
  
// Report 3  
let fleet: [Drone] = [WelderDrone(id: "W-9", cell: PowerCell(charge: 100))]  
let first = fleet[0]  
print(first.weldSeam())  
  
// Report 4  
protocol Labelled {  
    var componentID: String { get }  
}  
  
extension Labelled {  
    func label() -> String { "generic component" }  
}  
  
struct Thruster: Labelled {  
    let componentID: String  
    func label() -> String { "thruster \ \(componentID)" }  
}  
  
let parts: [Labelled] = [Thruster(componentID: "T-1")]  
print(parts[0].label())
```

For each: what the author expected, what actually happens (or why it won't build), the language rule, and a fix.

Report 3 needs a tool you haven't used yet. The array's element type is `Drone`, so the compiler only lets you call what a `Drone` has. To reach a subclass member you must ask at runtime with a conditional cast:

```
if let welder = first as? WelderDrone {  
    print(welder.weldSeam())  
}
```

Use it, and say in one sentence why `as?` returns an optional.

Report 4 is the interesting one. It compiles and prints `"generic component"`, not `"thruster T-1"`. Find out what makes the difference between a method the protocol *requires* and a method that only exists in its extension. One line changes the output.

Finale: The Mission Code

```
let missionCode = "\(A)-\(B)-\(C)-\(D)"
print("MISSION CODE: \(missionCode)")
```

- **A** — total work units (Level 3)
- **B** — total remaining drone charge (Level 3)
- **C** — drones that could still run one more task (Level 3)
- **D** — sum of all status codes across drones, sensors and the beacon (Level 5)

Every fragment computed, none typed in.

Defense Questions (0.5 points)

1. Drone satisfies mutating func recharge(by:) without writing mutating, and SensorModule cannot. Explain.
2. You used both a base class and a protocol in this assignment. Give one thing inheritance does that a protocol cannot, and one thing a protocol does that inheritance cannot.
3. What exactly does final prevent, and what did making runOnce() final protect you from?
4. In Report 4, why did the protocol extension's method win over the struct's own method?

Bonus (+1 point, explore on your own)

The lecture says abstraction means hiding details and showing only what matters. Our Drone base class leaks: Drone(id:cell:) can be instantiated directly, and a plain Drone does no work at all, which is meaningless for a fleet.

1. Swift has no abstract keyword. Find at least **two** different ways to make "you must not use Drone directly" visible — one that fails at runtime, one that fails at compile time.
2. Rebuild the fleet so that the base type is a protocol with a protocol extension instead of a class, and WelderDrone becomes a struct.
3. Write two or three sentences comparing the two designs. Which would you pick for this station, and what would change if the drones had to share mutable state?

Grading

Part	Points
Level 1 — Power Cell	1.5
Level 2 — The Fleet	2
Level 3 — The Shift	1.5
Level 4 — Diagnostics	2
Level 5 — Shared Behaviour	1.5
Level 6 — Incident Reports	1
Defense questions	0.5
Total	10
Bonus	+1
Each ! outside Level 6	−0.5
The Health Rule written more than once	−0.5
Wrong mission code	−1

One button, one screen. Make it work for hardware nobody has invented yet.